RCA Dashboard
Manager Explainer

How RCA Agents Work - Step by Step
Plain-language speaker script

Demo cells: XYZ401-XYZ410
Repo: github.com/aqil2020in/telecomgpt
Companion: docs/RCA_MANAGER_EXPLAINER.md

1. Opening (30 seconds)

Say: "Our RCA dashboard is like a network doctor. We already loaded sample network data for 10 demo cells. When I pick a cell, the system reads that data, calculates health numbers, runs expert checks, and tells us the most likely root cause and what to fix - with evidence, not guesswork."

2. Five layers (draw on whiteboard)

- Layer 1 - DATASETS (CSV): Raw events already in GitHub datasets/ folder
- Layer 2 - KPI SERVICE: Calculator - turns events into rates (87% HO success)
- Layer 3 - RULES: Telecom checklists (if metric > threshold, flag issue)
- Layer 4 - RCA AGENTS: Expert doctors (Handover, RLF, VoNR, Alarm...)
- Layer 5 - DASHBOARD: What you see - charts, findings, final report

Data flows UP: CSV -> KPIs -> Rules -> Agents -> Screen

3. Preloaded data - what to tell manager

We did NOT type data into the dashboard for each cell.

- 15 CSV files in GitHub: telecomgpt/datasets/
- Each row tagged with cell ID (XYZ401, XYZ402, ...)
- Your only input: pick a cell from the dropdown

Say: "Preloaded means demo data is shipped with the product. We only select the cell; the events are already there."

4. Example A - Handover page (easiest demo)

Show: Sidebar -> Handover -> Cell XYZ401

Step 1: 2_Handover.py opens the Handover screen.

Step 2: dashboard_utils.py gets data for XYZ401.

Step 3: loaders.py reads handover_events_enriched.csv from disk.

Step 4: kpi_service.py calculates HO success %, prep fail %, Xn fail %.

Step 5: Top of page = summary numbers. Middle = charts and event table (proof).

Step 6: HOAgent (specialists.py) runs ho_rules.py checklist on those KPIs.

Step 7: Bottom = findings: cause, confidence, evidence, recommended actions.

Say: "Handover page is one dataset, one specialist, one checklist - one clear diagnosis for mobility."

5. Example B - RCA Report (full team demo)

Show: RCA Report -> XYZ401 -> Handover failure -> Run Master RCA

Step 1: 8_RCA_Report.py - you ask: why is handover failing on XYZ401?

Step 2: kpi_service.py merges ALL CSVs into one health profile for the cell.

Step 3: rca_orchestrator.py calls many agents: HO, RLF, Alarm, Transport, ANR...

Step 4: Each agent runs its rules/*.py checklist on the same KPI summary.

Step 5: master_rca.py links related issues; orchestrator ranks findings.

Step 6: health_scoring.py gives overall score (e.g. 52/100).

Step 7: Screen shows: root cause, evidence, recommendations, confidence %.

Say: "RCA Report is all datasets, all specialists, team lead ranking - the final engineering answer with evidence and fix steps."

6. How it works WITHOUT Render

The Streamlit RCA dashboard does NOT need telecomgpt.onrender.com for demos. One Python process on your machine = dashboard + RCA engine together.

- Step 1: streamlit run dashboard/app.py
- Step 2: loaders.py reads CSV from datasets/ (disk, no network)
- Step 3: kpi_service.py calculates rates per cell
- Step 4: Pages show charts (proof) + metrics (summary)
- Step 5: agents + rules/*.py run local IF/THEN checklists
- Step 6: Findings on screen - no Render, no OpenAI required

Say: "Dashboard demo runs on this machine like Excel plus macros - Render is a separate door for chat and API, not needed for this demo."

Two paths, same RCA brain:

- Dashboard: Your PC -> Streamlit -> CSV -> Rules -> Screen (NO Render)
- Chat/API: Browser -> Vercel -> Render -> TNIC rules (uses Render)

Upload page ONLY can optionally call Render API; all sidebar pages stay local.

6b. What runs where - HOW and WHERE

Thing	Where	How
Streamlit UI	Your machine	streamlit run dashboard/app.py - one Python process
CSV data	Disk datasets/	loaders.py pd.read_csv - preloaded files
KPI calculation	Your machine	kpi_service.py counts rows, computes rates
RCA agents/rules	Your machine	specialists.py + rules/*.py IF/THEN checks
Handover/RLF/RCA	Your machine	pages/*.py -> utils -> agents -> screen

Handover trace: Browser -> 2_Handover.py -> loaders -> kpi_service -> HOAgent -> findings

NOT on your machine for demo: Render API, live OSS. OpenAI optional only.

Machines involved: 1. Cloud services: 0.

ARCHITECTURE diagram shows PRODUCTION (Render has RCA + data for chat).

Local Streamlit demo (Mode B) uses same code/CSVs on YOUR machine - both true.

7. Handover vs RCA Report

	Handover page	RCA Report
Question	How is HO?	Why is HO failing?
Data	Mainly HO CSV	All CSVs merged
Experts	1 Handover Agent	8-12 agents
Output	Few findings	Ranked root cause + report

8. Key Python files (if asked)

- Page UI: dashboard/pages/2_Handover.py or 8_RCA_Report.py
- Glue: dashboard/dashboard_utils.py
- Read CSV: datasets/loaders.py
- Calculate rates: datasets/kpi_service.py
- Experts: agents/specialists.py
- Checklists: rules/ho_rules.py, rlf_rules.py, ...
- Team lead: orchestrator/rca_orchestrator.py

9. Manager FAQ - quick answers

Q: Did we enter data manually?

A: No - preloaded CSV files in GitHub datasets/.

Q: Uses Render backend?

A: No for dashboard demo - local Streamlit + CSV + Python.

Q: Is it AI / ChatGPT?

A: Core is rule-based. AI optional for summary text only.

Q: OpenAI cost?

A: $0 unless narrative report ON + API key set.

Q: Can we trust it?

A: Every finding shows evidence and rule ID - auditable.

Q: Real network later?

A: Replace CSVs with real OSS exports; same agents/rules.

Q: Business value?

A: Faster RCA, one view across mobility/RF/voice/alarms.

10. Five-minute demo script

- 0-1 min: Home - show 10 cells XYZ401-410
- 1-2 min: Handover XYZ401 - metrics + charts
- 2-3 min: Scroll to HO Agent findings
- 3-4 min: RCA Report - Run Master RCA
- 4-5 min: Point at root cause, evidence, recommendations, confidence

11. One sentence to memorize

We preload demo network data in CSV files; when you pick a cell, the system summarizes it into KPIs, runs telecom expert rules through RCA agents, and delivers a ranked root cause with evidence, confidence, and fix steps - Handover shows one expert; RCA Report shows the full team.